

1. Find 10 different model providers and their models released in last 12 months

Model Provider	Model	Release Date	Cut-off Date	Cost of Tokens (Input / Output per 1M)	Context Window	Type	When to Use This Model
OpenAI	GPT-5.6 Sol	Jul 9, 2026	Feb 16, 2026	\$5.00 / \$30.00	1.05M (128K max output)	Closed frontier · reasoning · agents · computer use	Hardest coding, multi-step agents, cybersecurity, science, long docs
Anthropic	Claude Opus 5	Jul 24, 2026	May 2026	\$5.00 / \$25.00	1M (128K max output)	Closed frontier · adaptive thinking · agents	Complex coding, enterprise agents, tasks needing freshest Claude knowledge
Google DeepMind	Gemini 3.6 Flash	Jul 21, 2026	Mar 2026	\$1.50 / \$7.50	1M (65K max output)	Closed multimodal · agentic · coding	Fast agent loops, tool use, multimodal (image/audio/video), Google ecosystem
xAI	Grok 4.5	Jul 8, 2026	Feb 1, 2026	\$2.00 / \$6.00	500K	Closed reasoning · coding · vision	Agentic coding, Cursor/Grok Build, SWE tasks; strong token efficiency

Alibaba (Qwen)	Qwen3.8-Max	Aug 3, 2026	Not published	\$2.00 / \$6.00	1M (131K max output)	Closed/open-soon MoE · multimodal	Long-horizon coding, research, multimodal work; strong value vs Western flagships
Moonshot AI	Kimi K3	Jul 16, 2026	Not published	\$3.00 / \$15.00	1M	Open-weight MoE · multimodal · agents	Long-horizon agents, open deployment, frontier coding when you can self-host
DeepSeek	DeepSeek-V4-Flash	Apr 24, 2026	Not published	\$0.14 / \$0.28	1M	Open-weight MoE · agentic	Lowest-cost volume work, agents, long context on a budget
Cohere	Command A+	May 20, 2026	Apr 2025	~\$2.50 / \$10.00	128K (64K max output)	Open-weight MoE · enterprise · multilingual	Sovereign/on-prem deploy, EU multilingual RAG, private agentic workflows
Mistral AI	Mistral Large 3	Dec 2, 2025	Not published	\$0.50 / \$1.50	256K	Open-weight MoE · multimodal	Best open multimodal flagship at very low API cost
ByteDance	Seed-OSS-36B	Aug 2026	Not published	Free (self-host) / N/A API	512K	Open-weight · reasoning	On-prem research, Apache 2.0 deploy, long-context reasoning without API fees

2) Read 2–3 articles about how transformer works?

<https://www.datacamp.com/tutorial/how-transformers-work>

a) What Are Transformers?

Transformers were first developed to solve the problem of sequence transduction, or neural machine translation, which means they are meant to solve any task that transforms an input sequence to an output sequence. This is why they are called “Transformers”.

But let’s start from the beginning.

What Are Transformer Models?

A transformer model is a neural network that learns the context of sequential data and generates new data out of it.

To put it simply:

A transformer is a type of artificial intelligence model that learns to understand and generate human-like text by analyzing patterns in large amounts of text data.

Transformers are a current state-of-the-art NLP model and are considered the evolution of the encoder-decoder architecture. However, while the encoder-decoder architecture relies mainly on Recurrent Neural Networks (RNNs) to extract sequential information, Transformers completely lack this recurrency.

So, how do they do it?

They are specifically designed to comprehend context and meaning by analyzing the relationship between different elements, and they rely almost entirely on a mathematical technique called attention to do so.



Real-Life Transformer Models

BERT

Google's 2018 release of BERT, an open-source natural language processing framework, revolutionized NLP with its unique bidirectional training, which enables the model to have more context-informed predictions about what the next word should be.

LaMDA

LaMDA (Language Model for Dialogue Applications) is a Transformer-based model developed by Google, designed specifically for conversational tasks, and launched during the 2021 Google I/O keynote. They are designed to generate more natural and contextually relevant responses, enhancing user interactions in various applications.

GPT and ChatGPT

GPT and ChatGPT, developed by OpenAI, are advanced generative models known for their ability to produce coherent and contextually relevant text. GPT-1 was its first model launched in June 2018 and GPT-3, one of the most impactful models, was launched two years later in 2020.

Claude

Claude, developed by Anthropic, is a family of Transformer-based AI assistants designed with a focus on safety and helpfulness. Claude uses Constitutional AI (CAI), a training approach where the model is guided by a set of principles to produce helpful, harmless, and honest responses.

Modern Transformer Variants

Since the original 2017 architecture, Transformers have evolved significantly to address limitations and expand to new domains.

Vision Transformers (ViT)

[Vision Transformers](#) apply the self-attention mechanism to images by dividing them into patches and treating each patch as a token. This approach has proven highly effective for image classification, object detection, and image generation, often outperforming traditional CNNs on large datasets.

Multimodal Transformers

Modern models like GPT-5, Gemini 3, and Llama 4 process multiple input types (text, images, audio, video) within a single architecture. These multimodal Transformers use unified embedding spaces and cross-attention mechanisms to reason across different modalities simultaneously.

Efficient Transformers

The quadratic complexity of self-attention limits context length. Several innovations address this:

- [Flash Attention](#): Optimizes memory access patterns to reduce GPU memory usage and speed up training by 2-4x

- **Rotary Position Embeddings (RoPE):** Encodes position information through rotation matrices, enabling better extrapolation to longer sequences
- **Linear attention variants:** Linformer, Performer, and Longformer reduce complexity to $O(n)$ for processing very long documents
- **Mixture of Experts (MoE):** Activates only a subset of parameters per token, enabling larger models with similar compute costs

b) Vector Embedding:

https://aws.amazon.com/marketplace/build-learn/vector-databases-for-ai?trk=fa7e5851-0cc3-436a-b80c-6d80ff7394c2&sc_channel=ps&ef_id=Cj0KCQjw-MDTBhCgARIsAKAkdIS24zssUm0JsrMWIqZ_8HicBuy3kU7pRW0FLBFpPyW3SL-CGtxVLolaApuhEALw_wcB&gads_campaign=23958044345&gads_ag=198397576460&gads_ad=813473147185&gads_kw=vector%20database%20ai&gads_matchtype=p&gads_network=g&gads_device=c&gads_geo=9062046&gad_campaignid=23958044345&gbraid=OAAAAADjHtp8z06R0Tur8uj8lqIOitoDcy&gclid=Cj0KCQjw-MDTBhCgARIsAKAkdIS24zssUm0JsrMWIqZ_8HicBuy3kU7pRW0FLBFpPyW3SL-CGtxVLolaApuhEALw_wcB

RAG pipelines: Ground your LLM in your data

Retrieval-Augmented Generation feeds relevant documents to a model before it generates a response. The vector database is the storage and query layer. Given a user's question, it finds semantically similar chunks from your knowledge base, and hands them to the model. Without it, you're relying on the model's training data alone — outdated, or missing your domain entirely.

Agentic memory: Give your AI agent persistent context

AI agents that reason across multi-step tasks need memory that survives between sessions. Tool selections, conversation history, user preferences — these need semantic lookup, not just key-value retrieval. A vector database gives your agent the ability to recall relevant prior context. No replaying entire conversation histories.

Semantic search: Match intent, not just keywords

Traditional search returns results based on term frequency. Semantic search returns results based on meaning. A user searching “how to reduce cloud costs” should find your document titled “Optimizing AWS spend” — even though the words don't overlap. Vector storage makes this work by comparing embeddings rather than strings.

Vector databases available in AWS Marketplace

Two approaches: purpose-built vector databases designed from the ground up for vector workloads (Pinecone, Zilliz Cloud), and data platforms your team may already operate that add vector capabilities

(Elastic Cloud, Redis Cloud, MongoDB Atlas). We've seen both approaches ship to production — the right choice depends on whether you're building from scratch or extending what you have.

c) Feed forward Neural Network

<https://www.geeksforgeeks.org/deep-learning/feedforward-neural-network/>

Feedforward Neural Network (FNN) is a type of artificial neural network in which information flows in a single direction i.e from the input layer through hidden layers to the output layer without loops or feedback. It is mainly used for pattern recognition tasks like image and speech classification.

For example in a credit scoring system, banks use an FNN which analyze users financial profiles such as income, credit history and spending habits to determine their creditworthiness.

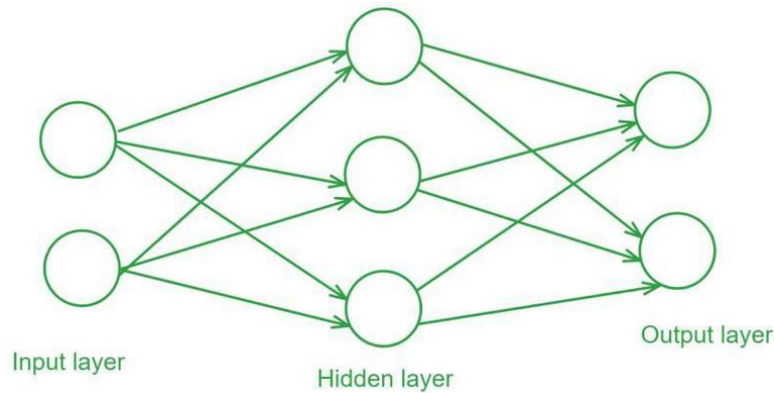
Each piece of information flows through the network's layers where various calculations are made to produce a final score.

Structure of a Feedforward Neural Network

Feedforward Neural Networks have a structured layered design where data flows sequentially through each layer.

1. **Input Layer:** The input layer consists of neurons that receive the input data. Each neuron in the input layer represents a feature of the input data.
2. **Hidden Layers:** One or more hidden layers are placed between the input and output layers. These layers are responsible for learning the complex patterns in the data. Each neuron in a hidden layer applies a weighted sum of inputs followed by a non-linear activation function.
3. **Output Layer:** The output layer provides the final output of the network. The number of neurons in this layer corresponds to the number of classes in a classification problem or the number of outputs in a regression problem.

Each connection between neurons in these layers has an associated weight that is adjusted during the training process to minimize the error in predictions.



Activation Functions

Activation functions introduce non-linearity into the network enabling it to learn and model complex data patterns.

Common activation functions include:

- **Sigmoid:** $\sigma(x) = \frac{1}{1+e^{-x}}$
- **Tanh:** $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- **ReLU:** $\text{ReLU}(x) = \max(0, x)$

Training a Feedforward Neural Network

Training a Feedforward Neural Network involves adjusting the weights of the neurons to minimize the error between the predicted output and the actual output. This process is typically performed using backpropagation and gradient descent.

1. **Forward Propagation:** During forward propagation the input data passes through the network and the output is calculated.
2. **Loss Calculation:** The loss (or error) is calculated using a loss function such as Mean Squared Error (MSE) for regression tasks or Cross-Entropy Loss for classification tasks.
3. **Backpropagation:** In backpropagation the error is propagated back through the network to update the weights. The gradient of the loss function with respect to each weight is calculated and the weights are adjusted using gradient descent.

Note: During training, a feedforward neural network performs forward pass followed by backpropagation to update weights, while during prediction only the forward pass is used.

